CAPE PENINSULA UNIVERSITY OF TECHNOLOGY

SDN260S SOFTWARE DESIGN II / GROUP 21

Marks Administration System

Core functionality demonstration, source review and team reflection

CONSOLE BUILD

Prepared 17 October 2026 / Version 1.0

PREPARED BY

Each member contributed 12.5% of the project.

Nhlonipho Sishi 12.5%
Database and domain architecture - schema, JDBC

Bopaki Montjane 12.5%
File processing and CSV engine - reader, writer

Scelo Mthethwa 12.5%
Database and domain architecture - domain model

Phakamile Tshenyego 12.5%
File processing and CSV engine - import service

Lucas Mfulwane 12.5%
Security and business logic - accounts, hashing

Hussein Akim 12.5%
User interface and quality assurance - Swing

Millano Redcliffe 12.5%
Security and business logic - marks, reports

Sinazo Radana 12.5%
User interface and quality assurance - CLI, tests

Contents

1 Scope and demonstration setup . . . 3

2 Working prototype demo . . . 4

3 UML alignment and source evidence . . . 8

4 Code review . . . 10

5 Challenges faced . . . 12

6 Requirements coverage and conclusion . . . 14

Appendix A Test and build evidence . . . 15

Appendix B Reproducing the demo . . . 15

Executive summary

Group 21 built a marks administration system for lecturers and heads of department. This document accompanies a console demonstration of the working prototype. It explains what the prototype does, points to the source code behind each result, and records the problems the team had to solve.

The demonstration was run on a clean database on 17 October 2026. A lecturer logs in, inspects a second-year civil engineering module, captures marks for a student and receives a weighted result of 83.25 (B+). The system then prints a student report with a GPA of 3.25, a subject report with class statistics and a missing-marks report. The same session exports 40 mark records to CSV. After the application is restarted, the marks are still in SQLite and the exported file imports again without creating duplicate students. A HoD login shows the extra user-management menu that a lecturer cannot reach.

The console build covers every core requirement in the brief: authentication, role-based access, persistent storage, weighted processing, reports and CSV import/export. Section 3 ties the running code to the class diagram, Section 4 reviews the code, and Section 5 is the account of what went wrong and who fixed it.

1 Scope and demonstration setup

This demonstration covers the console application only, as the brief requested. The Swing interface is in the build and is mentioned where it shares code with the console, but no GUI screens are shown. The console is the better demo for the marker because every action is repeatable: the same input file produces the same output, and the database can be inspected directly afterwards.

The prototype is built with Maven and runs as a single jar. The demonstration used a temporary database so that the repository data was left untouched; the same commands work against the default data/marks.db.

Component	Version	Role in the prototype
Java	17 (Temurin)	Language level and runtime
Maven	3.9	Build, dependency management, fat jar
SQLite JDBC	3.45.3.0	Embedded relational storage
OpenCSV	5.9	Reading and writing marks CSV files
JUnit 5	5.10.2	Automated tests (28 in total)

Table 1. Build and runtime dependencies.

Build and run

```
mvn clean package
java -Dmarksystem.db=/tmp/demo.db -jar target/marks-administration-system-1.0.0.jar
```

On first start the database is created and seeded with the CPUT Bellville engineering programmes, 31 modules, 23 students and 123 example marks. The seed runs once; a flag is stored in the settings table, so deleting all marks later does not bring the demo data back. Student numbers follow CPUT's nine-digit format, starting at 253243782.

A small input script drove the session so that the output could be captured consistently. The prompts and replies were taken from the recorded console log; box-drawing characters and tick marks were normalised to plain text for the PDF.

> *Not shown here: the Swing interface, network or multi-user access, and performance at faculty scale. The database is a local SQLite file, not a shared server.*

2 Working prototype demo

Each block below is copied from the session log. The comment underneath says what the result proves and where it comes from in the code.

2.1 Login and the role gate

Listing 1. Lecturer login and main menu (option 6 is absent).

```
Select interface mode:
  1. Command Line Interface (CLI)
  2. Graphical User Interface (GUI)
Enter choice (1 or 2): 1

=== Authentication Required ===
Username: lecturer1
Password: pass123
OK Logged in as: Dr. John Doe (LECTURER)
  Department: Civil Engineering

MAIN MENU
  1. Student Management
  2. Subject Management
  3. Mark Management
  4. Reports and Analytics
  5. CSV Import/Export
  0. Exit
```

The menu is built from the role stored with the account. A lecturer does not see the user-management option at all; typing 6 prints “This feature is available to HoD only.” A later login as hod1 adds option 6 and lists the three seeded accounts, which is the role check in User.hasAccess working through UserService.

2.2 Subject definition and assessment weights

Listing 2. CIV260S has three assessments that add up to 100%.

```
Enter subject code: CIV260S

Subject Details
Code: CIV260S
Name: Structural Analysis 2
Credits: 12
Description: Diploma in Engineering: Civil Engineering - Year 2 (CPUT Bellville)

Assessment Categories:
  Test 1               25.0%
  Test 2               25.0%
  Exam                 50.0%
```

Weights are stored per module, not hard-coded in the calculation. The service rejects a subject whose weights do not add up to 100%, so the numbers shown here are the numbers used in the next step.

2.3 Mark entry and weighted result

Listing 3. Entering three assessments for one student.

```
Enter student ID: 253243782
Enter subject code: CIV260S

Enter Marks for Hussein Akim - Structural Analysis 2
  Test 1 (Weight: 25%, 0-100, blank = missing): 82
  Test 2 (Weight: 25%, 0-100, blank = missing): 71
  Exam   (Weight: 50%, 0-100, blank = missing): 90

OK Marks entered successfully.
  Weighted Total: 83.25
  Grade: B+
```

The arithmetic is 82 x 0.25 + 71 x 0.25 + 90 x 0.50 = 83.25. The grade comes from the Mark.Grade thresholds (B+ starts at 80). Marks outside 0 to 100, or text, are rejected at the prompt; the service layer repeats the check so the GUI and CSV import cannot get around it.

2.4 Reports

Listing 4. Student report with the weighted score and GPA.

```
STUDENT MARK REPORT
Student ID: 253243782
Full Name: Hussein Akim
Program: Diploma in Engineering: Civil Engineering
Year: 2

Subject Code Subject Name                         Score % Grade
CIV260S      Structural Analysis 2                  83.25 B+
CIV270S      Concrete Technology 2                  75.60 B

Cumulative GPA: 3.25 / 4.00
```

Listing 5. Subject report: only the marks that exist are averaged.

```
Student ID Student Name              Score % Grade
253243782  Hussein Akim                83.25 B+
258070164  Sipho Ndlovu                71.75 C+
258543600  Thandiwe Moyo               57.25 F

Statistical Summary:
  Students with marks: 3 of 23
  Class Average: 70.75
  Highest Mark: 83.25
  Lowest Mark: 57.25

  Warning: Students with missing or absent marks: 20
```

The missing report that follows lists students with no mark and students with an incomplete entry separately for each module. Both reports use the same definition of missing, which was not true in the first build (see Section 5).

2.5 CSV export, restart and re-import

The export writes one row per student per module, with the assessment columns in the same order as the template. The computed WeightedTotal and Grade columns are included for reading in Excel; the importer ignores them and recalculates.

Listing 6. Export and the first rows of the file.

```
Enter output file path (e.g., marks_export.csv): /tmp/demo_marks.csv
OK Successfully exported 40 record(s) to /tmp/demo_marks.csv

StudentID,FirstName,LastName,Email,Program,Year,SubjectCode,SubjectName,
Assessment1,...,Assessment5,WeightedTotal,Grade
250660405,Ethan,van Wyk,250660405@mycput.ac.za,...,MEC260S,
Thermodynamics 2,82,81,70,64,,71.60,C+
```

The application was then stopped and started again with the same database. No marks were re-entered. The second session opened the same student and showed 83.25 and B+, which proves the write went to SQLite rather than an in-memory map. The exported file was imported back in that session; the summary reported 40 rows processed, 0 students created, 0 students updated and 40 marks saved. Re-importing the same file twice therefore updates instead of duplicating.

Listing 7. Re-import summary after restart.

```
Import complete.
Rows processed: 40
Students created: 0
Students updated: 0
Marks saved: 40
```

2.6 Database evidence

Listing 8. Schema and row counts after the session.

```
$ sqlite3 /tmp/demo.db ".tables"
marks   settings   students   subject_assessments   subjects   users

$ sqlite3 /tmp/demo.db "select count(*) from students;"
23
$ sqlite3 /tmp/demo.db "select count(*) from subjects;"
31
$ sqlite3 /tmp/demo.db "select count(*) from marks;"
123

lecturer1|LECTURER|pbkdf2$...
hod1|HOD|pbkdf2$...
```

Five business tables plus a settings table hold the data. Passwords are not stored in plain text; the suffix shows the PBKDF2 format now used for new accounts. Older accounts hashed with SHA-256 still log in (see the fifth problem in Table 6).

2.7 What the demo establishes

Area	Observed result	Main classes
Login and RBAC	Lecturer menu hides user management; HoD sees it	UserService, User, CLIMain
Persistent storage	23 students, 31 subjects, 123 marks; 83.25 survives restart	Database
Weighted processing	83.25 and B+ from 82/71/90 with 25/25/50 weights	Mark, MarkService
Reports	GPA 3.25; class average 70.75; 20 missing	ReportService
File I/O	40 records exported, 40 re-imported, no duplicates	CSVService, CSVHelper

Table 2. What the demo establishes.

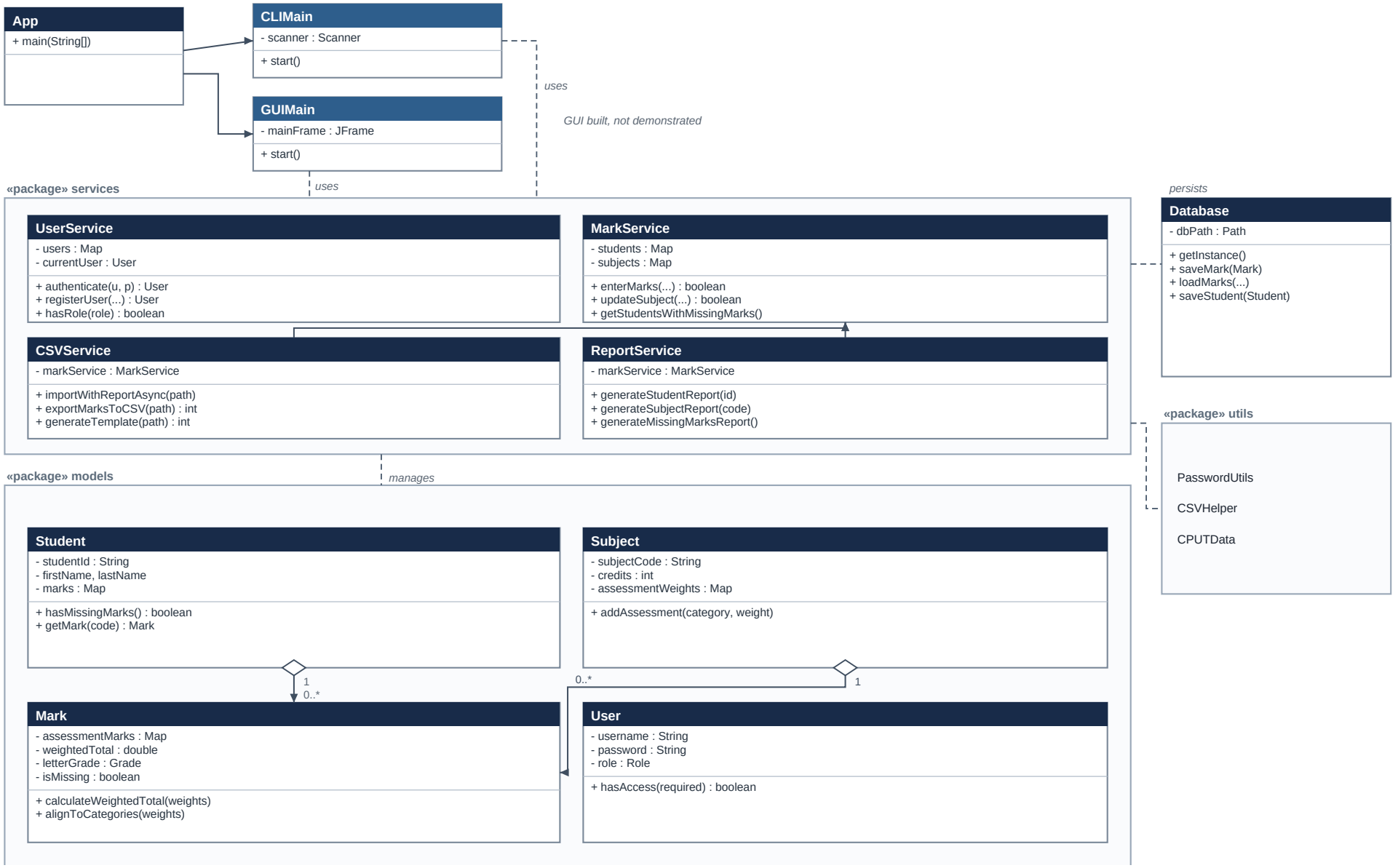


Figure 1. Class diagram of the submitted code. The GUI classes are shown for completeness but are not part of this demonstration; utility classes are grouped for legibility.

Database implements MarkRepository. UserService implements AuthenticationService. The console depends on those two interfaces.

3 UML alignment and source evidence

The implementation follows the four layers in the class diagram. Presentation classes call services; services hold the rules and use Database for persistence; Database maps rows to the model objects. The console and the Swing interface share exactly the same service objects, which is why a mark captured in the CLI appears in the GUI without extra code.

In the diagram, CLIMain and GUIMain depend on the service package as a whole, because each one uses several services. MarkService sits between the presentation and the database: it owns the in-memory maps of students and subjects and is the only class that changes a Mark. Student and Subject each hold zero or more Marks. CSVService and ReportService read and write through MarkService instead of touching the database directly, so file imports and reports apply the same validation and weighting rules as manual entry.

Class	Source file	Responsibility	Demo evidence
App	App.java	Entry point, CLI or GUI choice	Interface selection
CLIMain	cli/CLIMain.java	Menus, input, role gate	All console output
GUIMain	gui/GUIMain.java	Swing shell and tabs	Not shown
MarkService	services/MarkService.java	Students, subjects, marks, weights	83.25 / B+
UserService	services/UserService.java	Accounts, login, roles	lecturer1 and hod1
CSVService	services/CSVService.java	Import, export, template	40 records each way
ReportService	services/ReportService.java	Student, subject, missing reports	GPA 3.25, average 70.75
Database	db/Database.java	Schema, CRUD, transactions	23 / 31 / 123 rows
Student	models/Student.java	Student details and mark map	Hussein Akim
Subject	models/Subject.java	Module, credits, weights	25/25/50 split
Mark	models/Mark.java	Scores, total, grade, missing flag	83.25, B+
User	models/User.java	Account and LECTURER/HOD role	Dr. John Doe (LECTURER)
CSVHelper	utils/CSVHelper.java	CSV layout, template, export	Header and rows
CPUTData	utils/CPUTData.java	Seed programmes and students	23 students, 31 modules
PasswordUtils	utils/PasswordUtils.java	PBKDF2 hashing and verification	pbkdf2$ in users

Table 3. Class-to-code map for the classes in Figure 1.

3.1 Source excerpts

Listing 9. models/Mark.java (abridged): a missing assessment counts as zero and is flagged.

```
public void calculateWeightedTotal(Map<String, Double> weights) {
    double earned = 0, totalWeight = 0;
    boolean missing = false;
    for (Map.Entry<String, Double> entry : weights.entrySet()) {
        double weight = entry.getValue();
        totalWeight += weight;
        Double mark = assessmentMarks.get(entry.getKey());
        if (mark == null) missing = true;
        else earned += mark * (weight / 100.0);
    }
    weightedTotal = totalWeight > 0 ? (earned / totalWeight) * 100 : 0;
    isMissing = missing;
    letterGrade = determineGrade(weightedTotal);
}
```

Listing 10. services/MarkService.java (abridged): one entry point for marks, whatever the interface.

```
public boolean enterMarks(String studentId, String subjectCode, Mark mark, String enteredBy) {
    validateScores(mark);
    lock.writeLock().lock();
    try {
        Student student = students.get(studentId);
        Subject subject = subjects.get(subjectCode);
        if (student == null || subject == null) return false;
        mark.setEnteredBy(enteredBy);
        mark.alignToCategories(subject.getAssessmentWeights());
        database.saveMark(mark);
        student.addMark(subjectCode, mark);
        return true;
    } finally { lock.writeLock().unlock(); }
}
```

Listing 11. db/Database.java (abridged): marks are replaced in one transaction.

```
public void saveMark(Mark mark) {
    try (Connection conn = connect()) {
        conn.setAutoCommit(false);
        // DELETE FROM marks WHERE student_id = ? AND subject_code = ?
        // INSERT ... VALUES (?, ?, ?, ?, ?, ?, ?)   (batched per category)
        conn.commit();
    } catch (SQLException e) {
        conn.rollback();
        throw new DatabaseException("Could not save marks for " + mark.getStudentId(), e);
    }
}
```

Listing 12. services/UserService.java (excerpt): authenticate reads the account, verifies the password, then publishes the current user.

```
User user;
lock.readLock().lock();
try { user = users.get(username); }
finally { lock.readLock().unlock(); }

if (user == null || !PasswordUtils.verifyPassword(password, user.getPassword())) {
    return null;
}

lock.writeLock().lock();
try { currentUser = user; }
finally { lock.writeLock().unlock(); }
return user;
```

4 Code review

The code is organised by layer rather than by screen. Table 4 shows the package sizes; the counts exclude test code and HTML/README files.

Package	Files	Lines	Contents
models	4	509	Student, Subject, Mark, User
services	4	1 695	MarkService, UserService, ReportService, CSVService
db	1	675	Database schema, CRUD and transactions
utils	3	717	CSVHelper, CPUTData, PasswordUtils
cli	1	864	CLIMain menus and input
gui	12	1 921	Swing panels, dialogs and the main frame
App	1	75	Entry point
Total	26	6 456	Main source (tests: 781 lines in 7 classes)

Table 4. Main source size by package.

4.1 Naming, comments and structure

Class names are nouns (Student, MarkService), methods are verbs (enterMarks, calculateWeightedTotal) and fields are private. Public classes and the business methods carry Javadoc, 845 comment and Javadoc lines in the main source. Getters and setters are left without comments on purpose; they say what they do. The longest classes are MarkService (655 lines) and Database (675 lines). Both are split internally by section comments, but they are still the two classes a reviewer should read first.

4.2 Validation and error handling

Validation sits in the service layer, not in the menus. MarkService checks student names and years, subject credits, assessment weights (positive, at most 100 each, total 100) and every score (0 to 100, finite). UserService rejects empty usernames and passwords. CSV rows that fail validation become warnings in the import summary instead of aborting the whole file. Database errors are wrapped in a single DatabaseException with the operation and the student or subject in the message.

4.3 Threading and transactions

MarkService and UserService use ReentrantReadWriteLocks. Multi-step work such as a CSV import takes the write lock through withWriteLock, so a background import cannot interleave with a mark saved from the GUI. Every SQLite connection enables foreign keys and a five-second busy timeout; deletes and mark replacements run in a transaction with an explicit rollback on failure.

4.4 Security

New passwords use PBKDF2-HMAC-SHA256 with a random 16-byte salt and 120 000 iterations. Verification is constant time. Hashes created by the earlier build (salted SHA-256) are still accepted, and a malformed hash returns false instead of throwing. Role checks are enforced by the menus and by User.hasAccess, and the HoD cannot delete the account they are logged in with.

4.5 Tests

Test class	Tests	What it covers
MarkTest	5	Weights, missing assessments, category realignment
PasswordUtilsTest	5	PBKDF2, legacy hashes, corrupt input
MarkServiceTest	4	Category rename, validation, seed flag
DatabasePersistenceTest	6	Reload, delete, subject edits, users
CSVServiceTest	4	Template, import, export round trip, bad header
ReportServiceTest	3	Report columns, statistics, CSV export
MarkPanelTest	1	Existing marks preload into the entry form (headless)
Total	28	mvn test: 0 failures, 0 errors

Table 5. Automated test coverage.

4.6 What is still weak

The honest list, in the order we would fix it:

- MarkRepository is the persistence interface. Database is the SQLite implementation, so services save and load through that interface rather than through SQL. AuthenticationService is the login contract. UserService implements it, and the console menus depend on that interface.
- Database mixes schema creation, SQL and object mapping in one class. A small mapper per entity would shorten it and make the SQL easier to review.
- MarkService and Database are large. MarkService carries student, subject and mark operations that could be three services.
- The Swing panels refresh each other through a hand-written callback. An event bus or property change listener would be less fragile.
- CSV assessment columns are matched by position, not by category name. This is documented in the template, but renaming a column in Excel will shift marks.
- The demo accounts are seeded with known passwords. That is convenient for marking but must be removed or forced to change on first login in a real deployment.

> *The brief asks for well-commented classes, methods and interfaces. MarkRepository and AuthenticationService are those interfaces. Public classes and business methods carry Javadoc. The other items in this list are still open.*

5 Challenges faced

This is not a tidy retrospective. These are the problems that actually cost the team time, in the order they appeared, with the members who picked each one up. The split follows the ownership recorded in docs/WORK_DIVISION.md.

Problem	Who handled it	What was wrong and what changed
Database locking and a deadlocked report export	Nhlonipho, Hussein, Sinazo	CSV import ran on a background thread while the GUI read the same SQLite file, so a lock error was easy to hit. Worse, the report CSV export held its read lock and then called logActivity, which needs the write lock; the upgrade would deadlock on the first export. Added busy_timeout, moved the logging call after the read lock is released, and route imports through withWriteLock. ReportServiceTest now calls the export.
Renaming an assessment broke existing marks	Scelo, Millano	Changing Exam to Final Exam left the old scores in memory and in SQLite. A student who had 83.25 suddenly showed 75.00 with no missing flag, because the new 50% component was simply absent. Marks are now realigned to the current categories on edit and on load; stale rows are ignored and the database is rewritten to the new category list.
Missing assessments inflated the average	Millano, Lucas	The weighted total was normalised over the assessments that had been entered. A student missing the exam could still pass. The total now divides by the subject's full weight and sets isMissing, so 80/70 with no exam scores 37.5, not 75. Reports show the same number as the marks screen.
CSV round trip was not stable	Bopaki, Phakamile	Computed WeightedTotal and Grade columns, blank template rows and duplicate student/module rows caused confusing counts. Import now matches headers by name, maps assessment columns by position, counts a student/module pair once, and reports bad rows as warnings. The exported file imports without changes.
Old passwords versus the new hash format	Lucas	The first database used salted SHA-256. Moving to PBKDF2 would have locked out every existing account. Hashes now carry a version prefix; the verifier accepts both formats and rejects a corrupt hash with false instead of an exception.
Eight people, four pairs, one schema	All members	The handover between the CSV pair and the marks pair was the weak point: the importer depended on how MarkService aligned categories, and that contract changed twice during integration. The GUI and CLI were only tested together near the end, which exposed refresh and seed bugs. File ownership in WORK_DIVISION.md, pair reviews and the shared database tests reduced the rework.

Table 6. Problems, owners and what changed.

5.1 Each member's problem and the fix

Table 6 groups the work by problem. The table below is the same period seen from each member's side: what went wrong in the area they owned, what changed in the build we are submitting, and what they have not fixed yet. Nothing in the last column is done; those items are the next tasks, and they extend the list in Section 4.6.

Member and area	The problem they ran into	Fix in this build	Still open
Nhlonipho Sishi database, JDBC	A background CSV import and the GUI read the same SQLite file, which produced “database is locked” errors. The report export then took its read lock and called logActivity, which needs the write lock.	A five-second busy timeout on every connection, explicit commit and rollback in mark saves and deletes, logging moved after the read lock is released, and a seed-flag migration for databases created by the old build.	Database.java is 675 lines of DDL, SQL and mapping. Split it into per-entity mappers and add a concurrent import/read test.
Scelo Mthethwa domain model, seed	Renaming Exam to Final Exam left the old score in memory and in SQLite, so an 83.25 mark could drop to 75.00 with no warning. Deleting every mark also brought the demo data back on the next start.	alignToCategories rebuilds a mark's map from the subject's current weights on edit and on load, stale categories are ignored, missing flags live on each mark, and settings.seeded stops the reseed.	Model getters and setters still have no Javadoc, and there is no test that two fresh databases seed identical rows.
Lucas Mfulwane accounts, hashing	Moving from salted SHA-256 to PBKDF2 would have locked out every account already in the database. A malformed hash threw an exception during login, and currentUser was published outside the lock.	Hashes are versioned (pbkdf2$iterations$salt$hash), legacy SHA-256 still verifies, comparison is constant time, a corrupt hash returns false, currentUser is written under the write lock, and no one can delete their own account.	The demo accounts still use known passwords. There is no forced change on first login and no login throttling.

Member and area	The problem they ran into	Fix in this build	Still open
Millano Redcliffe marks, reports	The weighted total was normalised over the assessments that had been entered, so a student missing the 50% exam could still pass. The marks screen and the reports used different definitions of missing.	The total divides by the subject's full weight and sets isMissing, GPA and subject statistics read the same values, reports show one Score % column, and the report CSV export is wired to the button.	MarkService is 655 lines covering students, subjects and marks at once. Split it into three services with a shared repository.
Bopaki Montjane CSV layout, template	The exported file did not import cleanly: computed WeightedTotal and Grade columns, blank template rows and duplicate student/module rows produced wrong counts and shifted values.	One canonical header, assessment columns mapped by position, computed columns ignored on import, blank rows skipped, duplicate pairs counted once, and the template regenerated from the same layout.	Assessment columns are still matched by position. Renaming a column in Excel shifts marks; match by category name first, then fall back to position.
Phakamile Tshenyego import engine	The background import wrote marks while the GUI was saving them, one invalid row aborted the whole file, and the summary counted rows in a way that was easy to misread.	The import runs inside MarkService.withWriteLock, invalid rows become warnings instead of an abort, ImportSummary reports rows and pairs created, updated and saved, and failures are shown on the event thread.	No progress bar for large files and no retry after a failed import; the async path is only covered indirectly.
Hussein Akim Swing interface	Mark entry opened with blank fields, so saving one assessment wiped the others. Tabs showed stale data after an edit, the subject dialog added a phantom blank row, the report export button did nothing, and a user could delete their own account.	The form preloads the selected student's marks, rejects non-finite input, refreshAll notifies every tab after a save or import, dialogs rebuild their rows from the existing weights, export is wired, and the delete guard is in place. MarkPanelTest checks the preload headlessly.	Tabs talk through a hand-written refresh callback; it should be a property change listener. Only MarkPanel has a GUI test.
Sinazo Radana CLI, tests, docs	The CLI crashed instead of re-prompting when a year, credit value or mark was not a number. None of the fixed bugs had regression tests, and the README had drifted from the code.	Validation loops re-prompt at the prompt, seven test classes (28 tests) cover persistence, CSV, hashes, weighting, reports and the panel preload, and the README, DEMO and WORK_DIVISION files were reconciled.	CI runs mvn package only; the CLI menus are still checked with scripted sessions rather than JUnit.

Table 7. Each member's problem, the fix in this build and the next task.

5.2 Teamwork

The four role pairs from the design document worked as pairs rather than as eight independent streams. Nhlonipho owned the schema and Scelo owned the model, so the field mapping was reviewed by both before the services were written. Lucas and Millano split security from the marks engine but had to agree on what a missing mark means, because the GPA and the missing report read the same flag. Bopaki and Phakamile owned the two halves of the CSV path, which only worked once the template layout and the import mapping were treated as one contract. Hussein and Sinazo covered the GUI and the CLI plus QA, and did the cross-interface testing.

The part we underestimated was integration. Each pair could demonstrate its own layer, but the first full run showed that the layers disagreed about missing marks and about when marks should be reloaded. If we did this again we would put the end-to-end tests in place in week one, before the GUI existed, and treat the service signatures as frozen sooner.

5.3 What we would change

- Keep MarkRepository and AuthenticationService stable as new features are added.
- Run the database, CSV and report tests on every commit, not at the end.
- Split MarkService into student, subject and mark services; it is too easy for one change to affect three features.
- Use branches per pair and a short review before merging, instead of editing the same files from two sides.

6 Requirements coverage and conclusion

The brief asked for a working console prototype, at least 40 to 50% of the requirements, a code review and an account of the challenges. Table 8 maps the requirements to what was demonstrated.

Requirement	Status	Evidence in this demo
Login system	Complete	lecturer1 and hod1 log in; bad passwords fail
Role-based access	Complete	Lecturer cannot reach user management; HoD can
Data storage	Complete	SQLite tables, 23/31/123 rows, marks survive restart
Basic processing	Complete	83.25 and B+ from weighted assessments; GPA 3.25
File I/O	Complete	Template, 40-record export, 40-record re-import
Reports	Complete	Student, subject, missing and activity reports
Validation	Complete	Scores 0-100, weights total 100, years 1-10
Missing-entry flagging	Complete	20 students flagged in the subject report
Console interface	Complete	Every action in this document
GUI interface	Built, not demoed	12 Swing classes, headless panel test

Table 8. Requirements coverage.

The console build alone covers nine of the ten functional items, and the GUI is implemented on top of the same services. That is well above the 40 to 50% minimum. The demonstration deliberately stays on the console so the marker can follow each action and inspect the database afterwards.

The prototype is not a finished product. It is a stable demonstration of the core workflow: login, capture, calculate, persist, report and exchange data. The remaining gaps in Section 4.6 are real. The interface gap is closed:
Database and UserService implement them, as noted under Figure 1.

Appendix A Test and build evidence

The final build was made with the Maven wrapper-free command below. The shaded jar contains the SQLite driver and OpenCSV, so the marker only needs Java 17.

```
$ mvn -o clean package
...
[INFO] Tests run: 28, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Building jar: target/marks-administration-system-1.0.0.jar
```

Appendix B Reproducing the demo

Run each session against the same temporary database. The greater-than signs show menu navigation, not typed characters; the full prompts are shown in Section 2.

```
rm -f /tmp/demo.db
java -Dmarksystem.db=/tmp/demo.db -jar target/marks-administration-system-1.0.0.jar

Session 1 (lecturer): 1, lecturer1, pass123,
  2 > 3 > CIV260S > 0, 3 > 1 > 253243782 > CIV260S > 82 > 71 > 90,
  2 > 253243782, 3 > 253243782, 4 > CIV260S, 5, 0, 5 > 2 > /tmp/demo_marks.csv, 0, 0

Session 2 (persistence and re-import): 1, lecturer1, pass123,
  3 > 2 > 253243782, 0, 5 > 1 > /tmp/demo_marks.csv, 0, 6 (blocked), 0

Session 3 (HoD): 1, hod1, admin123, 6 > 1, 0, 0
```

The three sessions were recorded on 17 October 2026 and all exited with status 0. The database used for the recording was a temporary file; the repository's data/marks.db was not modified.